

Trout: Two-Round Threshold ECDSA from Class Groups

Hila Dahari-Garbman
Weizmann Institute of Science
Rehovot, Israel
hila.dahari@weizmann.ac.il

Ariel Nof
Bar-Ilan University
Ramat-Gan, Israel
ariel.nof@biu.ac.il

Luke Parker
Serai DEX
Orlando, Florida, USA
lukeparker@serai.exchange

ABSTRACT

We present Trout (Two-ROund Threshold), the *first* distributed two-round ECDSA signing protocol for arbitrary thresholds. Trout has constant upload bandwidth per-party and processing time linear in the amount of participants. Moreover, Trout achieves the Identifiable Abort (IA) property, which means that if the protocol cannot terminate due to a failure, parties can attribute the failure to a specific party. We achieve this without a trusted setup.

Our protocol relies on linear-homomorphic encryptions and commitments over class groups. To obtain our result, we leverage the recent construction of an exponent-VRP (Boneh et al., Eurocrypt 2025) and a novel protocol to multiply an encrypted value with a committed value and simultaneously decrypt it, which we call "scaled decryption". We believe that this protocol may be of independent interest.

Our protocol has a very low communication cost of just 6.5 KB sent per party. Furthermore, we implemented our protocol in Rust and provide benchmarks for various configurations, showing its practicality even for 100 parties. Our implementation includes a constant-time variant which, to the best of our knowledge, is the first of its kind for class-group-based threshold ECDSA protocols.

1 INTRODUCTION

The Elliptic Curve Digital Signature Algorithm (ECDSA) is one of the most popular digital signature schemes, mainly due to its use in the Bitcoin protocol and other cryptocurrencies. In the setting of *threshold* ECDSA, the private signing key is distributed across n parties with a threshold t , such that only if a subset of $t + 1$ parties exchange the information they hold, the key can be restored. To sign messages, any $t + 1$ parties need to run a secure protocol with the guarantee that only the generated signature is learned by the parties, while no information about the private key is leaked.

The problem of constructing efficient threshold ECDSA protocols have gained a lot of attention in the last decade or so, as multiple companies have used threshold-based solutions for building cryptocurrency wallets and custody [13, 23, 39]. The idea here is to distribute the key across multiple devices, making it much more challenging to steal the key, as it requires hacking into several devices. On the other hand, these devices need to interact each time a message is requested to be signed, in order to compute the signature. If the executed protocol is slow, this will have a devastating effect on performance. Hence, there is ongoing effort among the research community to develop threshold ECDSA protocols with high efficiency and throughput. The main difficulty stems from the fact that, unlike Schnorr signatures, ECDSA is not "MPC-friendly". Functionalities are considered MPC-friendly when they consist of linear operations. However, the ECDSA signature functionality requires computing k^{-1} in a distributed manner with no individual party knowing k , the signature's "nonce".

The first solution to any number of parties and any threshold was introduced in [27] and [3], but with an impractical distributed key generation protocol (since it requires distributed Paillier key generation for many parties). In 2018, concurrent works by Gennaro and Goldfeder [26], Lindell and Nof [31] and Doerner et al. [19] have presented the first full-threshold protocols for multiparty ECDSA, with practical distributed key generation and signing. Since then, a long line of works proposing new protocols have appeared, aiming to improve efficiency in various metrics, under different assumptions and cryptographic tools; see Table 1 for a detailed comparison. We note that there is also a line of works with protocols designed for the specific 2-party setting [2, 4, 9, 18, 30, 36, 37]. However, we focus on any number of parties and any threshold.

A particular important efficiency metric is the number of rounds in the protocol. Having a large number of rounds may result with a high latency protocol, making it too slow for many practical real-world scenarios. To the best of our knowledge, the smallest number of rounds to this date have been obtained in the recent work of Doerner et al. [20], who presented a signing protocol with three rounds of interaction. While the signing protocol of Abraham et al. [1] has two rounds, their protocol is not a threshold protocol, but a multiparty protocol. That is, their protocol requires all n parties to participate in the signing protocol, rather than a threshold $t < n$. This is problematic in many application, where some of the participants may not be always online. In addition, the protocol of [1] relies on a trusted setup to distribute PCG (pseudorandom correlation generator) seeds to the parties.

The question that arises is therefore: *is it possible to construct a two-round distributed signing protocol with a threshold key generation, that has low communication (without a trusted setup)?*

We note that two rounds seem to be the minimal number of rounds for threshold ECDSA, without techniques such as fully-homomorphic encryption, since the signature is a pair (r, s) where s is computed as a function of r . Hence, at least one round should be dedicated to computing r , and only then can the parties proceed to compute s , which requires at least one additional round.

1.1 Our Contribution

In this work, we answer this question in the affirmative. We present a new distributed ECDSA protocol, Trout, for n parties and any threshold $t < n$ which has the following features:

- The signing protocol requires only two rounds of interaction.
- Scalability: communication and local computation time grow linearly in the number of parties.
- It achieves the Identifiable Abort (IA) property, meaning that if the protocol aborts, then the parties can identify a corrupted party.

As an intermediate result, we also present a version of our protocol that achieves only security with abort. This version has the

Scheme	#Rounds	IA	Robust	Threshold	Per-Participant Upload	Premise
GG18[26]	8	✗	✗	t	$O(t)$	Paillier
LN18[31]	8	✗	✗	t	$O(t)$	Paillier/OT
CGMP[8]	4	✓	✗	t	$O(t)$	Paillier
ANOS22[1]	2	✗	✗	n	$O(n)$	PCG
HLNR18[29]	5	✗	✗	t	$O(t)$	OT
CCLST23[10]	7	✓	✗	t	$O(t)$	CL
WMYC23[35]	5	✓	✓	t	$O(n)$	CL
DKLs24[20]	3	✗	✗	t	$O(t)$	OT
WMC24[34]	4	✓	✓	t	$O(1)$	CL
FMM*24[25]	3	✓	✗	t	$O(1)$	Paillier
our work	2	✓	✗	t	$O(1)$	CL

Table 1: Comparison between distributed ECDSA signing protocols. IA = Identifiable Abort; Robust means that the protocol can terminate successfully as long as a threshold number of honest parties participate in each round (as opposed to non-robust threshold protocols where the *same* subset of $t + 1$ parties need to participate in each round); Threshold t means that the protocol allows signing by $t + 1$ parties only, whereas threshold n means that all n parties must be active in the signing protocol; Per-Participant Upload refers to the complexity of the unique messages each participant must upload to complete the protocol, with the security parameter omitted as a factor; OT = Oblivious Transfer; PCG = pseudorandom correlation generator; CL = Castagnos-Laguillaumie encryption scheme.

same number of rounds, but has less communication and processing time.

Our protocol relies on class groups hardness assumptions, as we make use of the linearly-homomorphic CL encryption scheme [11, 12] which works over class groups. We prove our protocol is secure via a simulation-based definition, under the hardness of the Hard Subgroup Membership Assumption (HSM) and the Adaptive Root Subgroup Assumption (ARSA).

Implementation and experimental results. We implemented our signing protocol in Rust. Our implementation is highly modular, and includes a variety of libraries for class groups, and (to the best of our knowledge) the first implementation of the DDH-based exponent VRF [4]. We ran extensive experiments to measure the running time of the different steps of our protocol. All the details can be found in Section 6. The communication cost of our protocol is 6345 bytes per party for the IA protocol, and 3902 bytes when relaxing security to security with abort only. The total running time for 2-out-of-3 signing is 959ms, whereas for 67-out-of-100 is just 2,664ms. We additionally provide an experimental constant-time implementation that is much slower, but provides resilience to side-channel attacks, reducing the likelihood of leaking bits of secrets. To the best of our knowledge, this is the first such implementation for threshold ECDSA over class groups.

1.2 Technical Overview

Denote by E a generator for an elliptic curve $\hat{E}$. Recall that the ECDSA signature for a message m is the pair (r, s) , where r is the x -coordinate of $R = k \cdot E$, k is a random nonce, and s is computed via the formula

$$s = k^{-1} \cdot (m' + r \cdot x)$$

where x is the signing key and $m' = \mathcal{H}(m)$ for some function $\mathcal{H}$.

Note that in the formula x is secret shared across the parties, whereas m' and r are public. Hence, it is clear that k should also remain secret. The challenge therefore is how to compute both r and k^{-1} over a secret k , shared across the parties. Our protocol follows the popular paradigm of having the parties compute $k \cdot u$ and $u \cdot (m' + r \cdot x)$ for a random u . Then, it is immediate to compute s by simply taking $(k \cdot u)^{-1} \cdot u \cdot (m' + r \cdot x)$.

It thus follows that a protocol that works according to the above approach has two main tasks: (1) computing r from the secret shared k and (2) computing two multiplications $k \cdot u$ and $u \cdot (m' + r \cdot x)$ over shared secret values k, u, x . In the following, we explain how our protocol addresses these two problems.

Computing r . Recall that r is the x -coordinate of $k \cdot E$ reduced into the scalar field of the elliptic curve. Hence, a protocol for computing r typically requires each party P_i to choose its random share k_i independently and then the parties interact to compute $k \cdot E$ where $k = \sum_{i=1}^n k_i$. The problem that arises here is that in order for the protocol to be simulatable, the simulator needs to know the shares of the corrupted parties before choosing the honest parties' shares. This is required since the simulator receives r as an input from the ideal functionality and needs to make sure that its interaction with the real-world adversary will yield the same r . Naively, this would require two rounds. In the first round, each party “commits” to its share in a way that hides its share but allows to extract the corrupted parties' shares. In the second round, each P_i reveals $k_i \cdot E$ and the parties compute $k \cdot E = \sum_{i=1}^n k_i \cdot E$. This enables the simulator to extract the corrupted parties' shares in the first round and only then choose shares for the honest parties that will yield the correct r in the second round. Unfortunately, computing r in two rounds will prevent us from achieving the main goal of computing the signature in two rounds, as computing r is only a prerequisite before performing the multiplication $u \cdot (m' + r \cdot x)$.

which depends on r . Hence, it is clear that obtaining a two-round signing protocol implies computing r in a single round.

To achieve this goal, we make use of a primitive called *exponent VRF* (verifiable random function), which was introduced by Boneh et al. [4]. Informally speaking, it enables a party P_i to publish $k_i \cdot E$, where k_i is an output of a pseudorandom function, with a key known to that party only. Specifically, party P_i commits to the key and then proves the correctness of $k_i \cdot E$, with respect to the committed key. This solves our problem, since now each party can commit to the pseudorandom function key in the ECDSA distributed key generation, and publish $k_i \cdot E$ together with a proof when signing a message. Since the computation in the signing protocol is now deterministic, the simulator knows the corrupted parties' expected messages and thus can simulate the correct computation of r .

We note that the authors of [4] have demonstrated how this primitive can be used to build a two-round two-party ECDSA. We adapt this tool also to the multiparty setting. As one of our contributions, we provide the first implementation of the DDH-based instantiation from [4], showing its practicality. For information about the implementation, including optimizations to [4] that we suggest, we refer the reader to Section 6.

Multiplying shared values and revealing the result. As we have seen, a main task in our signing protocol is to let the parties choose random secret values a and b , multiply them, and reveal the result. To this end, we make use of the CL linear-homomorphic encryption scheme [11, 12]. Naively, such a multiplication protocol would work as follows: (1) Each P_i chooses its shares a_i and b_i and publishes encryption $\text{Enc}(a_i)$. (2) The parties locally compute $\text{Enc}(a) = \sum_{i=1}^n \text{Enc}(a_i)$. Then, each party P_i locally computes $\text{Enc}(b_i \cdot a) = b_i \cdot \text{Enc}(a)$ and publishes the result. (3) The parties locally compute $\text{Enc}(a \cdot b) = \sum_{i=1}^n \text{Enc}(b_i \cdot a)$ and run a secure, one-round decryption protocol to obtain $a \cdot b$. Note that such a protocol would require three rounds of interaction, defeating our goal of obtaining a two-round signing protocol.

At a high level, our solution works by combining step 2 and 3 above. In our protocol, the parties use a statistically-hiding commitment over class groups for their shares of b , that already holds some information which is required for the decryption. As a result, they can scale $\text{Enc}(a)$ with b_i and decrypt $\text{Enc}(a \cdot b)$ in a single round. We call this protocol “scaled decryption”, since it is used to scale an encrypted value and decrypt the result. This protocol may be of independent interest and is described in Section 4.1.

Summarizing the above. Concluding our discussion so far, our signing protocol has the following two rounds. In the first round, the parties obtain their shares of k via invoking the evaluation function of the exponent VRF, publish them in-the-exponent as necessary to calculate r , and publish encryptions of their shares of k with commitments to their random shares of u . In the second round, the parties locally compute $\text{Enc}(k)$ and $\text{Enc}(m' + r \cdot x)$ and then run the scaled decryption protocol to compute $u \cdot k$ and $u \cdot (m' + r \cdot x)$ from which it is immediate to compute the signature s .

We note that the way the signing protocol is designed has immediate consequences on the key generation protocol. For example, the parties need to commit to their eVRF key, they need to compute an encryption of the private ECDSA key and so on. Overall, our

distributed key generation has three rounds, and is described in Section 3.

Avoiding expensive secret sharing over the integers. One advantage of our protocol worth mentioning is that we do not require threshold key generation for the CL encryption scheme. In particular, we only require the parties to generate a public encryption key, but the secret key does not have to be shared in any way. This is enabled since when decrypting ciphertexts in the signing protocol, the parties use the one-time randomness instead of the secret key. The advantage of this approach is that it avoids threshold key generation over unknown-order groups, which would require secret sharing over the integers [6]. Such schemes are expensive and computationally-wise do not scale well with the number of parties. By avoiding this, we achieve a highly efficient and scalable key generation protocol.

Achieving malicious security and IA. To enforce malicious behavior, we use zero-knowledge proofs. Each message in the protocol is accompanied with a proof that asserts its correctness. We note all the messages in our signing protocol are public and the same proof is sent to all parties. Hence, communication cost per party is constant. In addition, once a proof is rejected, the parties can abort while identifying the prover as the cheater.

We describe and prove our protocol in the hybrid model, where the ZK proofs are replaced with an ideal zero-knowledge functionality (such as, for example, in [31]). The zero-knowledge relations that we need are instantiated using the efficient proofs from [14].

As mentioned earlier, we present two variants for our protocol - one that achieves security with abort only and one that achieves IA. The difference between the two protocols is whether the second round's messages include ZK proofs or not. We observe that to achieve security with abort, it is not necessary to prove the correctness of the second round's messages. Instead, the parties can compute the signature optimistically, and use the ECDSA verification algorithm to verify its correctness. The reason this is possible is that given the encryptions and commitments in the first round, the second-round messages are in fact deterministic. Hence, in the security proof, the simulator knows the corrupted parties' expected messages (given that it extracts all encrypted and committed values in the first round) and can simulate the honest parties' accordingly.

2 PRELIMINARIES

Notations. We use lower-case letters to represent scalars, and upper-case letters to denote group elements. Variables used as randomness within ciphertexts/commitments are denoted with greek letters. We use $||$ to denote string concatenation. For a prime q , let $\mathbb{Z}_q$ denote a finite field of size q . We use κ to denote a fixed security parameter. We use $[n]$ to denote the set $\{1, \dots, n\}$.

Composing group elements is notated additively with scaling notated multiplicatively.

Security and network model. We assume a synchronous network with authenticated point-to-point channels between the parties. In the version of the protocol that achieves IA, we also assume a broadcast channel, which enables the parties to publish public messages.

We use the standard real/ideal world paradigm to prove the security of our protocols [7, 28]. In particular, for each of our protocol, we show that it realizes with malicious security an ideal functionality that we define later. We prove the security of our protocols in a hybrid model, where parties run a protocol with real messages and also have access to a trusted party computing a subfunctionality for them. The modular sequential composition theorem of [7] states that one can replace the trusted party computing the subfunctionality with a real secure protocol computing the subfunctionality. When the subfunctionality is g , we say that the protocol works in the g -hybrid model.

2.1 Groups and Assumptions

2.1.1 Elliptic Curves. Let Gen_{EC} be an algorithm that, given an input 1^κ , outputs the tuple

$$\text{ppEC} = (\hat{E}, q, E)$$

where $\hat{E}$ is a cyclic group with prime order q and E is a generator of $\hat{E}$. In this paper, the group operation is notated additively, and each point A in $\hat{E}$ is of the form $A = (a, \cdot)$ for some $a \in \mathbb{N}$.

2.1.2 Class Groups. Let Gen_{CL} be an algorithm that takes 1^κ and a prime order q as an input, and outputs the tuple

$$\text{ppCL} = (\bar{s}, \hat{G}, \hat{H}, G, H, G_q).$$

The set $\hat{G}$ with the addition operation is a finite abelian group of order $q \cdot s'$, where the length of s' is a function of κ and $\text{gcd}(q, s') = 1$. The value $\bar{s}$ is the upper bound of s' . One can decide if an element is in $\hat{G}$ in polynomial time. The set $\hat{H}$ with addition operation is the unique cyclic subgroup of $\hat{G}$ of order q , generated by H . The group $\hat{G}^q := \{q \cdot x, x \in \hat{G}\}$ is the subgroup of order s' of $\hat{G}$, generated by G_q . It thus holds that $\hat{G} = \hat{G}^q \times \hat{H}$. The discrete logarithm problem in $\hat{H}$ can be solved by a polynomial time algorithm DLsolve , i.e., for each $x \in \mathbb{Z}_q$ it holds that $x = \text{DLsolve}(x \cdot H)$.

Distributions. We define two distributions $\mathcal{D}_q$ and $\mathcal{D}$. Let $\mathcal{D}_q$ (resp., $\mathcal{D}$) denote the distribution which satisfies that $\{r \cdot G_q : r \leftarrow \mathcal{D}_q\}$ (resp., $\{r \cdot G : r \leftarrow \mathcal{D}\}$) is close to uniformly sampling random elements in $\hat{G}^q$ (resp., $\hat{G}$). For setting concrete parameters for these distributions, see [12].

Assumptions. The HSM assumption says that a PPT adversary cannot distinguish between random elements in $\hat{G}$ and $\hat{G}_q$.

DEFINITION 2.1 (HARD SUBGROUP MEMBERSHIP ASSUMPTION (HSM)). Let $\mathcal{A}$ be a PPT algorithm and consider the following experiment: The challenger runs $\text{Gen}_{\text{CL}}(1^\kappa)$ to obtain ppCL , samples x and x' from $\mathcal{D}_q$ and $\mathcal{D}$ respectively and $b \in \{0, 1\}$. Then, it sets $Z_0 = x \cdot G_q$ and $Z_1 = x' \cdot G$. $\mathcal{A}$ is given $(\text{ppCL}, Z_b, \text{DLsolve}())$ and outputs a bit b' . We say that $\mathcal{A}$ wins if $b' = b$. Denote by $\text{Adv}_{\mathcal{A}}^{\text{HSM}}(\kappa)$ the success probability advantage of $\mathcal{A}$ over an adversary who simply guesses b . We say that the HSM assumption is hard in G if for all PPT adversaries $\text{Adv}_{\mathcal{A}}^{\text{HSM}}(\kappa)$ is negligible in κ .

The Adaptive Root Subgroup Assumption says that an adversary cannot calculate U such that $\ell \cdot U = V$ where ℓ is an odd prime and V is chosen by the adversary before it knows ℓ .

DEFINITION 2.2 (ADAPTIVE ROOT SUBGROUP ASSUMPTION (ARSA)). Let $(\mathcal{A}_0, \mathcal{A}_1)$ be two PPT algorithms and consider the following experiment: the challenger runs $\text{Gen}_{\text{CL}}(1^\kappa)$ to obtain ppCL . Then, $\mathcal{A}_0$ is given ppCL to output (V, state) . The challenger then chooses a random $\ell \in \text{Primes}(\kappa)$. Then, $\mathcal{A}_1$ is given (ℓ, state) and outputs U . We say that $(\mathcal{A}_0, \mathcal{A}_1)$ win if $V = \ell \cdot U$ and $q \cdot U$ is not the identity. Denote by $\text{Adv}_{\mathcal{A}}^{\text{ARSA}}(\kappa)$ the success probability of $(\mathcal{A}_0, \mathcal{A}_1)$. We say that the ARSA assumption is hard in G if for all PPT adversaries $\text{Adv}_{\mathcal{A}}^{\text{ARSA}}(\kappa)$ is negligible in κ .

2.2 The ECDSA signature

The ECDSA signature scheme is computed over the elliptic curve $\hat{E}$. Let $\mathcal{H} : \{0, 1\}^* \rightarrow \hat{E}$. Then, the scheme consists of the following three algorithms:

KeyGen(1^κ): Choose $x \in \mathbb{Z}_q$. The output is a pair (x, X) , where $X = x \cdot E$. The signing key is x while X is the verifying key.

Sign($x; m$): Given a message m , choose a random k and set $R = k \cdot E$. Let r be the x -coordinate of $R \bmod q$. Output (r, s) , where $s = k^{-1} \cdot (\mathcal{H}(m) + r \cdot x)$

Verify($X; m, \sigma$): Given a message m and a signature $\sigma = (r, s)$, check $0 < r, s < q$, set $z = ((\mathcal{H}(m) \cdot s^{-1}) \cdot E) + ((r \cdot s^{-1}) \cdot X)$ and check that z is not the point at infinity and its x -coordinate modulo q is equal to r . If it holds, output 1. Otherwise, output 0.

The multiparty ECDSA ideal functionality. We next present an ideal functionality for multiparty computation of the signature as in [31]. Note that the ECDSA functionality allows the ideal-world adversary to see the generated signature but then force the honest parties to output another signature (r, s') as long as it verifies w.r.t. the message m . This is a generic model that captures also the version of ECDSA which does not have the strongly-unforgeable property. Clearly, when the strongly-unforgeable version of ECDSA is used, the adversary cannot produce a different signature for m .

FUNCTIONALITY 2.1 ($\mathcal{F}_{\text{ECDSA}}$ - DISTRIBUTED ECDSA).

Parties: $P_1, \dots, P_N$ and the ideal-world adversary $\mathcal{S}$.

Parameters: $t \leq N \in \mathbb{N}$ and $\kappa \in \mathbb{N}$.

- Operation:
- Upon receiving $(\text{sid}, \text{KeyGen})$ from all n parties:
 - (1) Let $(x, X) \leftarrow \text{KeyGen}(1^\kappa)$.
 - (2) Store (sid, x) .
 - (3) Send (sid, X) to all parties.
 - Upon receiving $(\text{sid}, \text{Sign}, m)$ from $t + 1$ parties:
 - (1) Retrieve (sid, x)
 - (2) Send $(\text{sid}, (r, s) = \text{Sign}(x, m))$ to $\mathcal{S}$.
 - (3) Wait to receive back (r, s') from $\mathcal{S}$. If $\text{Verify}(X; m, (r, s')) = 1$ output (r, s') to all parties. Otherwise, output abort.
-

The above functionality achieves security with abort only. We also consider a stronger notion of Identifiable Abort (IA). Let $\mathcal{F}_{\text{ECDSA}}^{\text{IA}}$ be an ideal functionality defined as in Functionality 2.1 with one change: if $\mathcal{S}$ outputs an incorrect signature, then it must send an index i of a corrupted party to the trusted party computing $\mathcal{F}_{\text{ECDSA}}^{\text{IA}}$. Then, the functionality sends (abort, i) to the honest parties and halts.

2.3 Class Group Encryption

We use the CLT encryption scheme over class groups [12]. Recall that we work over a group $\hat{G} = \hat{G}_q \times \hat{H}$, with a generator G_q for $\hat{G}_q$ and a generator H for $\hat{H}$. The order of $\hat{G}_q$ is unknown while the order q of $\hat{H}$ is known. The decisional Diffie-Hellman problem is hard in $\hat{G}_q$ while in $\hat{H}$ the discrete log problem is easy.

KeyGen: Given 1^k and the class group public parameters ppCL, the algorithm outputs a pair $(Y^{\text{cl}}, y^{\text{cl}})$ where $y^{\text{cl}} \in \mathcal{D}_q$ is randomly chosen and $Y^{\text{cl}} = y^{\text{cl}} \cdot G_q$. The private key y^{cl} is used for decryption while Y^{cl} is the public encryption key.

Encrypt: Given a message m and the encryption key Y^{cl} , choose a random $\alpha \in \mathcal{D}_q$ and output

$$\text{Enc}_{Y^{\text{cl}}}(\alpha, m) = (\alpha \cdot G_q, \alpha \cdot Y^{\text{cl}} + m \cdot H)$$

Decrypt: Given a ciphertext $\tilde{C} = (C_1, C_2)$ and a decryption key y^{cl} , output

$$\text{Dec}_{y^{\text{cl}}}(\tilde{C}) = \text{DLsolve}(C_2 - (y^{\text{cl}} \cdot C_1))$$

Note that $m \cdot H = \tilde{C}_2 - y^{\text{cl}} \cdot \tilde{C}_1 \in \hat{H}$ and thus we can apply the DLsolve algorithm to find m . Another way to decrypt is using the randomness instead of the decryption key, i.e., compute $\text{DLsolve}(C_2 - \alpha \cdot Y^{\text{cl}})$. That is, it is possible to decrypt using the one-time randomness without knowing the private key. Looking ahead, our protocols will use this simple fact to run secure distributed decryption without relying on a decryption key.

2.3.1 Homomorphic Operations. The above scheme allows performing linear operations locally. Let $\tilde{A} = (\tilde{A}_1, \tilde{A}_2) = \text{Enc}_{Y^{\text{cl}}}(\alpha, m_1)$ and $\tilde{B} = (\tilde{B}_1, \tilde{B}_2) = \text{Enc}_{Y^{\text{cl}}}(\beta, m_2)$ be two ciphertexts. In addition, let $s \in \mathbb{Z}_q$ be a scalar. We define the following local operations:

- $\tilde{A} + \tilde{B}$: $(\tilde{A}_1 + \tilde{B}_1, \tilde{A}_2 + \tilde{B}_2) = \text{Enc}_{Y^{\text{cl}}}(\alpha + \beta, m_1 + m_2)$
- $s \cdot \tilde{A}$: $(s \cdot \tilde{A}_1, s \cdot \tilde{A}_2) = \text{Enc}_{Y^{\text{cl}}}(s \cdot \alpha, s \cdot m_1)$
- $\tilde{A} + s$: $(\tilde{A}_1, \tilde{A}_2 + S) = \text{Enc}_{Y^{\text{cl}}}(\alpha, m_1 + s)$, where $S = s \cdot H$.

2.4 Class Group Commitments

We use Class Group commitments, with G_q for the randomness and Y^{cl} for the value. That is, a commitment to $m \in \mathcal{D}_q$ is computed by choosing a random $\alpha \in \mathcal{D}_q$ and outputting

$$\text{Com}(\alpha, m) = \alpha \cdot G_q + m \cdot Y^{\text{cl}}.$$

The commitment scheme is statistically hiding and computationally binding *as long as the sender does not know the discrete logarithm of Y^{cl} with respect to G_q* [15]. Since in our protocol, none of the parties knows the secret key y^{cl} , this property holds. Moreover, our protocol does not even require a threshold key generation for Y^{cl} , as decryption is carried out using the one-time randomness. Hence, the parties can choose Y^{cl} to be a uniform element in the group $\hat{G}_q$ and use it in the commitment scheme. See also Section 2.6.

2.5 Shamir Secret Sharing and Feldman's VSS

The Shamir Secret sharing [33] with a threshold t consists of two algorithms: Share and Reconstruct. In the Share algorithm, a dealer wishes to share a secret $x_0 \in \mathbb{Z}_q$ to parties $P_1, \dots, P_N$. In the algorithm, the dealer chooses a random polynomial $f(x) = a_0 + \sum_{j=1}^t a_j x^j$.

x^j of degree t , under the constraint that $f(0) = a_0 = x_0$. Then, it hands each party P_i the point $f(i)$. In the Reconstruct algorithm, the parties send their shares to each other and use them to compute x_0 via Lagrange interpolation. We denote by $\mathcal{L}_i^S(x)$ the Lagrange coefficient that corresponds to a subset S of size $t+1$, a party $i \in S$ and an evaluation point x .

Feldman's VSS [21] enables the parties to detect cheating in the sharing process. To this end, in the sharing procedure, the dealer publishes also the polynomial coefficients in the exponent. That is, the dealer sends each party P_i its share $f(i)$ and also publishes $A_j = a_j \cdot E$ for $j = 0, \dots, t$. Hence, each party can check that its share s_i satisfies the equation $s_i \cdot E = \sum_{j=0}^t i^j \cdot A_j$. Formally, we define two algorithms:

- $((s_1, \dots, s_N), \{a_j\}_{j=1}^t, \{A_j\}_{j=0}^t) \leftarrow \text{Vss.Share}(x_0)$: In this algorithm, the dealer runs $\text{Share}(x_0)$ to get the polynomial coefficients $a_0, \dots, a_t \in \mathbb{Z}_q$ (where $a_0 = x_0$), and then computes $A_j = a_j \cdot E$ for each $j \in \{0, \dots, t\}$.
- $\beta \leftarrow \text{Vss.Vrfy}(s_i, \{A_j\}_{j=0}^t)$: In this algorithm, party P_i checks that $s_i \cdot E = \sum_{j=0}^t i^j \cdot A_j$. If the equation holds, it outputs 1. Otherwise, the output is 0.

2.6 Class Group Key Generation

In our protocol, we make use of class group encryptions and commitments where the public encryption key Y^{cl} is generated jointly by the parties, such that none of the parties know the secret decryption key. The ideal functionality $\mathcal{F}_{\text{CLKeyGen}}$ is formalized in Functionality 2.2. We stress that the ideal functionality is not required to share the secret key between the parties, as our protocol does not require threshold decryption at any stage. It only needs to sample a random element in the group and hands it to the parties as the public key. Since we use Y^{cl} in the statistically-hiding commitment scheme defined in Section 2.4 as a random element in the group, this functionality cannot be realized using just any protocol, e.g., the biased key generation protocol in [6]. Nevertheless, any secure coin tossing protocol to sample a random element in the group suffices.

FUNCTIONALITY 2.2 ($\mathcal{F}_{\text{CLKeyGen}}$ - CL KEY GENERATION).

The functionality works with N parties and an ideal world S who controls a subset of parties $T \subset [N]$.

Upon receiving the command $(\text{sid}, \text{KeyGen})$ from all parties, if sid is not stored, then $\mathcal{F}_{\text{CLKeyGen}}$ chooses a random $y^{\text{cl}} \in \mathcal{D}_q$ and computes $Y^{\text{cl}} = y^{\text{cl}} \cdot G_q$. Then, it hands Y^{cl} to the parties, and store $(\text{sid}, Y^{\text{cl}})$.

2.7 eVRF

An *exponent Verifiable Random Function* (eVRF) [4] is a primitive that takes x as an input and outputs $Y = y \cdot E$ together with a proof that $y = F(k, x)$ where $F(k, \cdot)$ is a keyed function whose output is indistinguishable from random elements in its range. We define the ideal functionality $\mathcal{F}_{\text{eVRF}}$ to compute an eVRF as in [4]. The functionality consists of two stages. In the initialization stage,

the keyed function is being set. Then, in the evaluation stage, the function is evaluated over an input x .

FUNCTIONALITY 2.3 ($\mathcal{F}_{\text{eVRF}}$ - EXPONENT VRF).

The functionality works with parties $P_1, \dots, P_n$.

- Upon receiving (init, i) from some party P_i :
 - If P_i is honest, then receive sid from the ideal world adversary. If sid has not been stored before, then store (sid, i).
 - If P_i is corrupt, then receive also (sid, f), where f is a description of a deterministic polynomial-time computable function. If sid has not been stored before, then store (sid, i, f).
 - Upon receiving (Eval, i , sid, x) from P_i :
 - If P_i is honest:
 - If (sid, i) is stored and a tuple (sid, i, x, y, Y) does not exist, then choose $y \leftarrow \mathbb{Z}_q$, compute $Y = y \cdot E$ and store (sid, i, x, y, Y). Then, send (Eval, i , sid, x, y, Y) to P_i and (Eval, i , sid, x, Y) to all parties.
 - If P_i is corrupt:
 - If (sid, i, f) is stored, then compute $Y = f(x) \cdot E$ and send (Eval, i , sid, x, Y) to all parties.
-

2.8 Zero-Knowledge Ideal Functionalities and Relations

We define a zero-knowledge proof of knowledge ideal functionality for any relation R in Functionality 2.4.

FUNCTIONALITY 2.4 ($\mathcal{F}_{\text{zk}}^R$ - ZERO-KNOWLEDGE PROOF OF KNOWLEDGE FOR RELATION R).

The functionality works with parties $P_1, \dots, P_n$.

Upon receiving (sid, x, w) from P_i , if sid has not been used by P_i before, send (sid, x, β) to all parties where $\beta = 1$ if $R(x, w) = \text{true}$ and $\beta = 0$ otherwise.

We realize the functionality using the proofs of [14] (which is an improvement of [38]). These proofs have been shown to be secure under the Adaptive Root Subgroup Assumption and Hard Subgroup Membership Assumption. For completeness, in Appendix A we present a general protocol, based on [14], that captures all the relations listed below. ZK proofs over class groups can be complicated, since they require computing an inverse in a group of unknown order. Earlier works [9] used a single-bit challenge, which required many repetitions of the proof to obtain a sufficiently small soundness error, thereby making ZK proofs for class groups slow and expensive. The works of [14, 38] were able to overcome this problem by using two different challenges over prime fields, significantly improving both the proof size and the computation time while maintaining the ability to extract the witness. This opened the door to practical and concretely-efficient threshold signing protocols using class groups, as in this work.

2.8.1 Relations. We list several instantiations which we use in the protocol.

- Knowledge of discrete logarithm over an elliptic curve

$$R_{\text{Dlog}} = \{(A_0, \dots, A_t, (a_0, \dots, a_t)) : \forall j \in \{0, \dots, t\} : A_j = a_j \cdot E\}$$

- Knowledge of CL plaintext

$$R_{\text{CLKwlg}} = \{(\tilde{C}, (\alpha, a)) : \tilde{C} = \text{Enc}(\alpha, a) = (\alpha \cdot G_q, \alpha \cdot Y^{\text{cl}} + a \cdot H)\}$$

- Knowledge of committed value

$$R_{\text{ComKwlg}} = \{(B, (\beta, b)) : B = \text{Com}(\beta, b) = (\beta \cdot G_q + b \cdot Y^{\text{cl}})\}$$

- Equality between a CL encrypted value and EC discrete log

$$R_{\text{CL-EC}} = \{(A, \tilde{C}, (\alpha, a)) : A = a \cdot E \wedge \tilde{C} = \text{Enc}(\alpha, a)\}$$

- Computing affine operations over commitments using other committed values

$$\begin{aligned} R_{\text{affCom}} &= \{(F, C, B, A, (\alpha, \beta, b)) : \\ &F = b \cdot A - \alpha \cdot B + \beta \cdot C \wedge C = \alpha \cdot G_q \\ &\wedge B = \text{Com}(\beta, b)\} \end{aligned}$$

2.8.2 The Committed Zero-Knowledge Functionality. We also define an ideal functionality which allows the parties to commit to their non-interactive ZK proof before revealing it to the other parties. The formal description appears in Functionality 2.5. It can be realized using any secure string commitment.

FUNCTIONALITY 2.5 ($\mathcal{F}_{\text{com-zk}}^R$ - THE COMMITTED NIZK FUNCTIONALITY FOR RELATION R).

The functionality works with parties $P_1, \dots, P_n$.

- Upon receiving (sid, comProve, x, w) from P_i : if sid has not been used before by P_i , store (sid, comProve, i, x, w) and send (sid, comProve, i) to the other parties.
 - Upon receiving (sid, decompProof) from P_i retrieve (sid, comProve, i, x, w) and send (sid, decompProof, x, β) to all parties, where $\beta = 1$ if $R(x, w) = \text{true}$ and $\beta = 0$ otherwise.
-

3 KEY GENERATION PROTOCOL

In this section, we present a protocol to generate the ECDSA keys. At the end of the protocol, the parties should hold the ECDSA public key $X = x \cdot E$, without any of the parties knowing the secret signing key x . Instead, x is shared across the parties via Shamir's secret sharing. In addition, the key generation should output some additional information that is used later in the signing protocol. This includes a public key encryption key for the class group encryption and an encryption of the ECDSA secret key using this scheme (more accurately, the protocol outputs an encryption of each party's secret share, from which an encryption of the secret key can be locally computed). In addition, the parties initialize the eVRF functionality that is later called in the signing protocol.

The protocol is formally described in Protocol 3.1. The parties use the ideal functionality $\mathcal{F}_{\text{CLKeyGen}}$ to generate the public encryption key and Feldman's VSS to generate the ECDSA public key. Then, each party locally computes their share of the secret key and send an encryption of the resulting share accompanied with a ZK proof.

To prevent biasing the key, we require the parties to commit to the polynomial coefficients they chose in the first round and reveal it only in the second round.

PROTOCOL 3.1 (DISTRIBUTED KEY GENERATION).

- **Public input:** $\text{sid}, \text{ppEC}, \text{ppCL}$
 - **The protocol:** Each party P_i works as follows:
 - **Round 1:**
 - (1) Send (init, i) to $\mathcal{F}_{\text{eVRF}}$.
 - (2) Send $(\text{sid}, \text{KeyGen})$ to $\mathcal{F}_{\text{CLKeyGen}}$ to receive Y^{cl} .
 - (3) Choose a random x_i and run $\text{Vss.Share}(x_i)$ to receive $s_{i,1}, \dots, s_{i,N}, \{a_{j,i}\}_{j=1}^t, \{A_{j,i}\}_{j=0}^t$.
 - (4) Send $(\text{sid}, \text{comProve}, \{A_{j,i}\}_{j=0}^t, (x_i, \{a_{j,i}\}_{j=1}^t))$ to $\mathcal{F}_{\text{com-zk}}^{R_{\text{Dlog}}}$.
 - **Round 2:** Upon receiving $(\text{sid}, \text{comProve}, k)$ for each $k \in [N]$:
 - (1) Send $(\text{sid}, \text{decomProof})$ to $\mathcal{F}_{\text{com-zk}}^{R_{\text{Dlog}}}$.
 - (2) Send $s_{i,k}$ to party P_k .
 - **Round 3:** Upon receiving $(\text{sid}, \text{decomProof}, \{A_{j,k}\}_{j=0}^t, \beta)$ from $\mathcal{F}_{\text{com-zk}}^{R_{\text{Dlog}}}$ for each $k \in [N]$ and $s_{k,i}$ from each P_k :
 - (1) If $\beta = 0$ for any $k \in [N]$, then send abort to the other parties and halt.
 - (2) For each $k \in [N]$: run $\text{Vss.Vrfy}(s_{k,i}, \{A_{j,k}\}_{j=0}^t)$ to receive β . If $\beta = 0$, send abort to the other parties and halt.
 - (3) Set $X = \sum_{k=0}^N A_{0,k}$, $x_i = \sum_{k=1}^N s_{k,i}$ and $X_i = \sum_{k=1}^N (A_{0,k} + \sum_{j=0}^{t-1} A_{j,k} \cdot k^j)$.
 - (4) Choose a random $\delta_i \in \mathcal{D}_q$ and compute $\tilde{C}_i = \text{Enc}(\delta_i, x_i)$.
 - (5) Send $(\text{sid}, X_i, C_i, (\delta_i, x_i))$ to $\mathcal{F}_{\text{zk}}^{R_{\text{CLKwlg}}}$.
 - **Computing the output:** Upon receiving $(\text{sid}, X_k, C_k, \beta)$ from $\mathcal{F}_{\text{zk}}^{R_{\text{CLKwlg}}}$ for all $k \in [N]$, if $\beta = 0$ for some k , then send abort to all parties and abort the protocol. Otherwise, Output $X, Y^{\text{cl}}, \{X_k\}_{k=1}^N, \{C_k\}_{k=1}^N, x_i$.
-

Protocol with public VSS. In our protocol, the parties distribute the shares using private point-to-point channels. This is the only private message sent in the protocol. It is possible to make the protocol work with public messages only, by having each party P_i encrypt the share intended to P_k and publish the ciphertext. For example, one can use the Class Group here. Each party would distribute their own encryption key (an element of the Class Group) which enables other parties to publish ciphertexts only the intended recipient can decrypt.

4 SIGNING PROTOCOL

4.1 Scaled Decryption

We begin with a subprotocol that is used in our main protocol. The protocol computes a multiplication of an encrypted value a with a committed value b , where both a and b are additively shared across the parties. Naively, such a protocol would take at least two rounds: in the first round, the parties will multiply their shares of b with $\text{Enc}(a)$ and send the result to the other parties. In the second round, the parties sum the messages from the first round to obtain $\text{Enc}(a \cdot b)$. Then, the parties interact to decrypt $a \cdot b$. Our protocol avoids the second round interaction by additionally preparing the decryption material in the first round, to enable local decryption once the parties obtain $\text{Enc}(a \cdot b)$. The protocol is formally described in Protocol 4.1. We prove its correctness in Claim 4.1. Note that our

protocol has two versions. Ver-A is the basic version that suffices to achieve security with abort in our main protocol. Ver-IA is used in our version that achieves the stronger notion of security with identifiable abort.

PROTOCOL 4.1 (SCALED DECRYPTION).

- **Public input:** $\text{sid}, \text{ppCL}, Y^{\text{cl}}, \{B_j\}_{j=1}^n, \{\tilde{A}_j\}_{j=1}^n$
- **Private input of each P_i :** $\alpha_i, \beta_i, b_i \in \mathcal{D}_q$ and $a_i \in \mathbb{Z}_q$ such that

$$\tilde{A}_i = \text{Enc}(\alpha_i, a_i), \quad B_i = \text{Com}(\beta_i, b_i)$$

- **The protocol:** Let

$$\tilde{A} = (A_1, A_2) = \sum_{j=1}^n \tilde{A}_j = \text{Enc}(\alpha, a), \quad B = \sum_{i=1}^n B_i = \text{Com}(\beta, b)$$

Each party P_i does the following:

- (1) Compute:
 - $F_i = b_i \cdot A_2 - \alpha_i \cdot B + \beta_i \cdot A_1$
 - (2) Send:
 - **Ver-A:** F_i to all the other parties
 - **Ver-IA:** $(\text{sid}, F_i, B, A_1, A_2, B_i, (\alpha_i, \beta_i, b_i))$ to $\mathcal{F}_{\text{zk}}^{R_{\text{affCom}}}$
 - (3) Upon receiving:
 - **Ver-A:** F_j from each party P_j
 - **Ver-IA:** $(\text{sid}, F_j, B, A_1, A_2, B_j, \beta)$ from $\mathcal{F}_{\text{zk}}^{R_{\text{affCom}}}$ for all $j \in [n]$.
- When Ver-IA is used: If $\beta = 0$ for some $j \in [n]$, then send abort to the other parties and halt.
- Compute:
- $F = \sum_{j=1}^n F_j$.
 - $c = \text{DLsolve}(F)$

- **Output:** c
-

CLAIM 4.1. *If all parties act honestly, then $c = a \cdot b$.*

PROOF. To prove the claim holds, we need to show that $F = (a \cdot b) \cdot H$. This holds since

$$\begin{aligned} F &= \sum_{i=1}^n F_i = \sum_{i=1}^n (b_i \cdot A_2 - \alpha_i \cdot B + \beta_i \cdot A_1) = \\ &= \sum_{i=1}^n (b_i \cdot \alpha \cdot Y^{\text{cl}} + b_i \cdot a \cdot H - \alpha_i \cdot \beta \cdot G_q - \alpha_i \cdot b \cdot Y^{\text{cl}} + \beta_i \cdot \alpha \cdot G_q) \\ &= a \cdot b \cdot H \end{aligned}$$

as required. $\square$

4.2 The Signing Protocol

In this section we present our signing protocol. The version we present in this section achieves only security with selective abort. In the next section, we show how to augment the protocol to achieve the stronger notion of security with identifiable abort. Recall that the signature is $s = k^{-1} \cdot (\mathcal{H}(m) + r \cdot k)$ where r is the x-coordinate of $k \cdot E$ reduced into the scalar field. As in previous works, we leverage Beaver's trick to compute the signature. Specifically, the parties compute $u \cdot k$ and $u \cdot (\mathcal{H}(m) + r \cdot x)$ in parallel, reveal them, and then each party can locally compute $s = (u \cdot k)^{-1} \cdot (u \cdot (\mathcal{H}(m) + r \cdot x))$ which equals to $k^{-1} \cdot (\mathcal{H}(m) + r \cdot k)$.

To perform this in two rounds, we use the $\mathcal{F}_{\text{eVRF}}$ functionality to produce $k \cdot E$ in a single round. In addition, each party commits

to its share of u and encrypts its share of k in the first round. Thus, at the beginning of the second round, the parties can compute $\text{Enc}(k)$, $\text{Com}(u)$ and $\text{Enc}(\mathcal{H}(m) + r \cdot x)$. It thus remains to call the scaled decryption protocol (Protocol 4.1) twice, on $\text{Enc}(k)$, $\text{Com}(u)$ and on $\text{Enc}(\mathcal{H}(m) + r \cdot x)$, $\text{Com}(u)$. The two executions ends with the parties holding $u \cdot k$ and $u \cdot (\mathcal{H}(m) + r \cdot x)$ which suffices for computing the signature as explained above. Thus, the protocol has two overall rounds. The formal description appears in Protocol 4.2.

PROTOCOL 4.2 (DISTRIBUTED SIGNING).

Let $\mathcal{H} : \{0, 1\}^* \rightarrow \hat{E}$.

The protocol is executed by a set of parties S indexed by $\{\ell_1, \dots, \ell_n\}$ (s.t. $n = t + 1 \leq N$). In the protocol, the notation P_i is used to denote party P_{ℓ_i} .

- **Public input:** sid , ppEC , ppCL , m , Y^{cl} , X , $\{X_j\}_{j=1}^n$, $\{\tilde{C}_j\}_{j=1}^n$
 - **Private input of each P_i :** x_i, δ_i such that $X_i = \mathcal{L}_{\ell_i}^S(0) \cdot x_i \cdot E$ and $\tilde{C}_i = \mathcal{L}_{\ell_i}^S(0) \cdot \text{Enc}(\delta_i, x_i)$.
 - **The protocol:**
 - **Round 1:**
 - (1) Send $(\text{Eval}, i, \text{sid}, m)$ to $\mathcal{F}_{\text{eVRF}}$ to receive back k_i and compute $R_i = k_i \cdot E$
 - (2) Choose random α_i, β_i, u_i and compute:
 - * $\tilde{K}_i = \text{Enc}(\alpha_i, k_i) = (\alpha_i \cdot G_q, \alpha_i \cdot Y^{\text{cl}} + k_i \cdot H)$.
 - * $U_i = \text{Com}(\beta_i, u_i) = \beta_i \cdot G_q + u_i \cdot Y^{\text{cl}}$.
 - Send:
 - (a) $(\text{sid}, R_i, \tilde{K}_i, (\alpha_i, k_i))$ to $\mathcal{F}_{\text{zk}}^{\text{RCL-EC}}$
 - (b) $(\text{sid}, U_i, (\beta_i, u_i))$ to $\mathcal{F}_{\text{zk}}^{\text{RComKwlg}}$
 - **Round 2:**
 - (1) Upon receiving:
 - * $(\text{Eval}, \text{sid}, j, m, R_j)$ from $\mathcal{F}_{\text{eVRF}}$
 - * $(\text{sid}, R_j, \tilde{K}_j, \beta)$ from $\mathcal{F}_{\text{zk}}^{\text{RCL-EC}}$
 - * (sid, U_j, β) from $\mathcal{F}_{\text{zk}}^{\text{RComKwlg}}$
 for all $j \in [n]$, if $\beta = 0$ for some $j \in [n]$, then send abort to the other parties and halt. Otherwise, compute:
 - * $R = \sum_{j=1}^n R_j$. Let r be the x-coordinate of R reduced into $\mathbb{Z}_q$.
 - * $\tilde{Z}_j = r \cdot \tilde{C}_j$ for all $j \neq 1$ and $\tilde{Z}_1 = \mathcal{H}(m) + r \cdot \tilde{C}_1$.
 - (2) Run Protocol 4.1 (using Ver-A) on $(\text{sid}, \text{ppCL}, Y^{\text{cl}}, \{\tilde{U}_j\}_{j=1}^n, \{\tilde{K}_j\}_{j=1}^n)$ to receive $u \cdot k$
 - (3) Run Protocol 4.1 (using Ver-A) on $(\text{sid}, \text{ppCL}, Y^{\text{cl}}, \{\tilde{U}_j\}_{j=1}^n, \{\tilde{Z}_j\}_{j=1}^n)$ to receive $u \cdot (\mathcal{H}(m) + r \cdot x)$.
 - (4) Compute $s = (u \cdot k)^{-1} \cdot u \cdot (\mathcal{H}(m) + r \cdot x)$.
 - (5) Run $\text{Verify}(X; m, (r, s))$ to receive β . If $\beta = 0$, then send abort to the other parties and halt.
 - **Output:** (r, s)
-

An optimization: working with r^{-1} to reduce computational costs. In Round 2, the parties need to locally multiply r with the ciphertext $\tilde{C}_j$ for each $j \in [n]$, which requires performing expensive operations over the class group. We can avoid this cost using the following idea: the parties will perform scalar multiplication of $\mathcal{H}(m) \cdot r^{-1}$, and add the result to $\tilde{C}_1$, where $\tilde{Z}_j$ is simply $\tilde{C}_j$ for all $j \neq 1$. This means that at the end of the protocol, the parties obtain $s' = k^{-1}(\mathcal{H}(m) \cdot r^{-1} + x)$. Now, the parties can correct the result, by multiplying s' with r . Note that yet again the operation is scalar multiplication and not a group operation. This replaces the cost of two scalings of class group elements with one inversion over a finite field and one multiplication over a finite field.

4.3 Proof of Security

In this section, we prove the security of our protocol. We note that the proof here refers to the Ver-A in Protocol 4.1 (scaled decryption). We prove the following theorem:

THEOREM 4.3. *Let $t \in [N]$ be a security threshold. Then, under the hardness of the HSM assumption, Protocol 3.1 (distributed key generation) and Protocol 4.2 (distributed signing) securely compute $\mathcal{F}_{\text{ECDSA}}$ with abort in the $(\mathcal{F}_{\text{eVRF}}, \mathcal{F}_{\text{CLKeyGen}}, \mathcal{F}_{\text{com-zk}}^{\text{RDlog}}, \mathcal{F}_{\text{zk}}^{\text{RCLKwlg}}, \mathcal{F}_{\text{zk}}^{\text{RComKwlg}}, \mathcal{F}_{\text{zk}}^{\text{RAffCom}}, \mathcal{F}_{\text{zk}}^{\text{RCL-EC}})$ -hybrid model, in the presence of malicious adversaries controlling up to t malicious parties.*

PROOF. Denote by C the set of corrupted parties, and by j^* an index of an honest party. We describe a simulator to the protocol, and prove that the simulated and real execution are computationally indistinguishable. In the simulation, the simulator plays the role of the ideal functionalities $\mathcal{F}_{\text{eVRF}}$, $\mathcal{F}_{\text{CLKeyGen}}$, $\mathcal{F}_{\text{com-zk}}$ and $\mathcal{F}_{\text{zk}}$ and the honest parties. The simulator will simulate the role of the honest parties as in a real execution (by following the protocol's instructions) except for the honest party P_{j^*} , for which the simulation will be different from the real execution.

Key Generation. We construct a simulator S_{kg} for the key generation protocol as follows. The simulator S_{kg} receives the ECDSA key X from the trusted party computing $\mathcal{F}_{\text{ECDSA}}$ and invokes the real-world adversary $\mathcal{A}$ on $(\text{sid}, \text{ppEC}, \text{ppCL})$:

- **Round 1:**
 - (1) Playing the role of $\mathcal{F}_{\text{eVRF}}$, S_{kg} receives from $\mathcal{A}$ the message (init, i, f) for each $i \in C$.
 - (2) Playing the role of $\mathcal{F}_{\text{CLKeyGen}}$, S_{kg} chooses a random $y^{\text{cl}} \in D_q$, and computes $Y^{\text{cl}} = y^{\text{cl}} \cdot G_q$. Then, it hands $\mathcal{A}$ the public key Y^{cl} .
 - (3) For every honest P_i (but P_{j^*}), S_{kg} chooses a random x_i and runs $\text{VSS.Share}(x_i)$ to receive $s_{i,1}, \dots, s_{i,N}$, $\{a_{j,i}\}_{j=1}^t$, $\{A_{j,i}\}_{j=0}^t$ according to the protocol instructions.
 - (4) S_{kg} plays the role of $\mathcal{F}_{\text{com-zk}}^{\text{RDlog}}$, and receives from $\mathcal{A}$ the message $(\text{sid}, \text{ComProve}, \{A_{j,i}\}_{j=0}^t, (x_i, \{a_{j,i}\}_{j=1}^t))$ for each $i \in C$, and extracts $(x_i, \{a_{j,i}\}_{j=1}^t)$.
 - (5) S_{kg} sets P_{j^*} first round values by following the instructions:
 - (a) Set $A_{0,j^*} = X - \sum_{k=0, k \neq j^*}^N A_{0,k}$.
 - (b) Choose random $s_{j^*,k} \in \mathbb{Z}_q$ for each $k \in C$.
 - (c) For each $k \in C$ define the equation

$$A_{0,j^*} + \sum_{\ell=1}^t A_{\ell,j^*} \cdot k^\ell = s_{j^*,k} \cdot E$$

where $\{A_{\ell,j^*}\}_{\ell=1}^t$ are the missing values. Then, find an assignment for each A_{ℓ,j^*} such that all equations are satisfied.

Note that there at most t such equations, and so S_{kg} can find a solution that satisfies this set of equations.

- **Round 2:**
 - (1) S_{kg} plays the role of $\mathcal{F}_{\text{com-zk}}^{\text{RDlog}}$, and hands $\mathcal{A}$ the message $(\text{sid}, \text{ComProve}, k)$ for every honest party P_k ($k \notin C$).

- (2) S_{kg} sends $\mathcal{A}$ all shares $s_{k,i}$ from each honest party P_k to each corrupted party P_k ($k \in C$) as chosen in the previous round.
- **Round 3:**
- (1) For each honest party P_k , S_{kg} plays the role of $\mathcal{F}_{com-zk}^{R_{Dlog}}$ and sends the message $(sid, decomProof, \{A_{j,k}\}_{j=0}^t, 1)$ to $\mathcal{A}$.
- (2) For each $k \in C$ (i.e., each corrupted party), if the values sent by $\mathcal{A}$ to $\mathcal{F}_{com-zk}^{R_{Dlog}}$ are incorrect, i.e., there exists $j \in \{0, \dots, t\}$ such that $A_{j,k} \neq a_{j,k} \cdot E$, S_{kg} simulates $\mathcal{F}_{com-zk}^{R_{Dlog}}$ sending $(sid, decomProof, \{A_{j,k}\}_{j=0}^t, 0)$, sends abort to $\mathcal{F}_{ECDSA}$, simulates the honest parties aborting the execution, and outputs whatever $\mathcal{A}$ outputs. Otherwise, it sends the message $(sid, decomProof, \{A_{j,k}\}_{j=0}^t, 1)$.
- (3) S_{kg} plays the role of the honest parties running $Vss.Vrfy(s_{k,i}, \{A_{j,k}\}_{j=0}^t)$ for each $k \in C$ to receive β . If $\beta = 0$ in any of the executions, S_{kg} sends abort to $\mathcal{F}_{ECDSA}$, simulates the honest parties aborting the execution, and outputs whatever $\mathcal{A}$ outputs.
- (4) For each $i \in C$, S_{kg} computes x_i and X_i :
- $X_i = \sum_{k=1}^n (A_{0,k} + \sum_{j=1}^{t-1} A_{j,k} \cdot k^j)$ (as in protocol's instructions).
 - Note that according to the protocol description, we have

$$x_i = \sum_{k=1}^N s_{k,i} = \sum_{k \in C} s_{k,i} + \sum_{k \in [n] \setminus C} s_{k,i}.$$

Now, the right values are the shares sent from S_{kg} to $\mathcal{A}$ when playing the role of the honest parties, and hence $\mathcal{S}$ can compute $\sum_{k \in [n] \setminus C} s_{k,i}$. To compute $\sum_{k \in C} s_{k,i}$ which consists of shares sent from corrupted parties to corrupted parties, S_{kg} uses the polynomial coefficients it extracted. That is, S_{kg} first computes $s_{k,i} = x_k + \sum_{j=1}^t a_{j,k} \cdot i^j$ for each $k, i \in C$ and then compute $\sum_{k \in C} s_{k,i}$.

- (5) For every honest party P_i , S_{kg} computes $X_i = \sum_{k=1}^N (A_{0,k} + \sum_{j=1}^{t-1} A_{j,k} \cdot k^j)$ (as in protocol's instructions).
- (6) For every honest party P_i , S_{kg} chooses a random δ_i and computes $\tilde{C}_i = \text{Enc}(\delta_i, 0)$.
- (7) S_{kg} plays the role of $\mathcal{F}_{zk}^{R_{CIKwlg}}$ sending $(sid, X_i, \tilde{C}_i, 1)$ to $\mathcal{A}$ for each i such that P_i is honest.
- (8) S_{kg} plays the role of $\mathcal{F}_{zk}^{R_{CIKwlg}}$ and receives from $\mathcal{A}$ the message $(sid, X_i, \tilde{C}_i, (\delta_i, x_i))$ for each $i \in C$. If there exists $i \in C$ such that $X_i \neq x_i \cdot E$ or $\tilde{C}_i \neq \text{Enc}(\delta_i, x_i)$, S_{kg} sends abort to the trusted party computing $\mathcal{F}_{ECDSA}$, simulates the honest parties aborting the protocol, and outputs whatever $\mathcal{A}$ outputs. Otherwise, it extracts (δ_i, x_i) .
- **Output:** S_{kg} outputs $Y^{\text{cl}}, X_i, \tilde{C}_i, \delta_i$ for each $i \in [N]$ and x_i for each $i \in C$.

We argue that the simulation and the real-world execution are computationally indistinguishable. Observe that a corrupted party P_k 's view in the protocol consists of random Shamir shares $s_{k,i}$, $\{A_{j,i}\}_{j=0}^t$ and $\tilde{C}_i$ for each honest party P_i . Recall that the adversary sees at most t shares which are uniformly distributed. This holds for both the simulation and real execution, and so the t shares

are distributed the same in both executions. Next, in both executions $\{A_{j,i}\}_{j=0}^t$ are uniformly distributed under the constraint that $Vss.Vrfy$ will output 1. Finally, in Round 3, in the real execution P_k receives the encryptions of x_i , whereas in the simulation it receives the encryption of 0. However, from the security of the encryption scheme, P_k cannot distinguish between the two cases. We thus conclude that the key generation protocol is secure.

Signing. We proceed to the distributed signing protocol.

Simulating the scaled decryption protocol. We begin by constructing a sub-simulator for the scaled decryption protocol.

- $\mathcal{S}_{sd}^{\text{online}}$ works as follows:

– **Inputs:**

- * $\{B_j\}_{j=1}^n, \{\tilde{A}_j\}_{j=1}^n$.
- * For each $i \in C$: $\alpha_i, \beta_i, a_i, b_i \in D_q$ such that $\tilde{A}_i = \text{Enc}(\alpha_i, a_i)$ and $B_i = \text{Com}(\beta_i, b_i)$.
- * For every $j \in [n] \setminus \{C \cup \{j^*\}\}$: α_j, β_j, b_j such that $A_{i,1} = \alpha_i \cdot G_q$ and $B_i = \text{Com}(\beta_i, b_i)$.
- * $a \cdot b$

– **Simulation:**

- (1) For each $i \in [n] \setminus \{j^*\}$: $\mathcal{S}_{sd}^{\text{online}}$ computes $F_i = b_i \cdot A_2 - \alpha_i \cdot B + \beta \cdot A_1$.
- (2) $\mathcal{S}_{sd}^{\text{online}}$ computes $F_{j^*} = a \cdot b \cdot H - \sum_{i \in [n] \setminus \{j^*\}} F_i$.
- (3) $\mathcal{S}_{sd}^{\text{online}}$ sends F_j for each $j \in [n] \setminus C$ to $\mathcal{A}$.
- (4) Upon receiving $F_{i,j}$ for each $i \in C$ intended to P_j from $\mathcal{A}$ (recall that in this step the parties send point-to-point messages), $\mathcal{S}_{sd}^{\text{online}}$ computes for each honest party P_j : $C_j = \sum_{i \in C} F_{i,j} + \sum_{k \in [n] \setminus C} F_k$ and run $\text{DLsolve}(C_j)$ to receive c_j .

Simulating the signing protocol. Now, we are ready to present the simulator $\mathcal{S}$ for the entire signing protocol. The simulator $\mathcal{S}$ receives (r, s) from the trusted party computing $\mathcal{F}_{ECDSA}$, from which it computes $R = k \cdot E$. In addition, it holds $(Y^{\text{cl}}, X, \{x_i\}_{i \in C})$ and the encryptions of $\{x_i\}_{i \in C}$ (from the key generation). The simulator $\mathcal{S}$ uses $\mathcal{S}_{sd}^{\text{online}}$ as a sub-simulator. The simulator $\mathcal{S}$ works as follows:

- **Input:** $sid, m, s, R, Y^{\text{cl}}, \{X_i\}_{i=1}^n, \{\tilde{C}_i\}_{i=1}^n, \{x_i\}_{i \in C}, \{\delta_i\}_{i \in [n] \setminus \{j^*\}}$.
- **Simulating the first round:**

- (1) Playing the role of $\mathcal{F}_{eVRF}$, $\mathcal{S}$ computes k_i for each $i \in C$ and $R_i = k_i \cdot E$. This is possible since $\mathcal{S}$ knows from the key generation for every corrupted P_i , the deterministic function f_i that P_i uses to compute $k_i = f_i(m)$. Hence, $\mathcal{S}$ can compute k_i for all the corrupted parties.
- (2) To compute the R_j held by honest parties: $\mathcal{S}$ chooses random $k_j \in \mathbb{Z}_q$ for each $j \in [n] \setminus \{C \cup \{j^*\}\}$ and computes $R_j = k_j \cdot E$. Then, $\mathcal{S}$ computes $R_{j^*} = R - \sum_{i \in [n] \setminus \{j^*\}} R_i$.
- (3) For each honest party P_j such that $j \neq j^*$: $\mathcal{S}$ chooses $\alpha_j \in D_q$ and computes $\tilde{K}_j = \text{Enc}(\alpha_j, k_j)$. For P_{j^*} , $\mathcal{S}$ chooses a random $K_{1,j^*} \in \hat{G}_q$ and $K_{2,j^*} \in \hat{G}$ and sets $\tilde{K}_{j^*} = (K_{1,j^*}, K_{2,j^*})$.
- (4) For each honest party P_j such that $j \neq j^*$: $\mathcal{S}$ chooses random β_j, u_j and computes $U_j = \text{Com}(\beta_j, u_j)$. For P_{j^*} , $\mathcal{S}$ chooses a random $U_i \in \hat{G}_q$.
- (5) Playing the role of $\mathcal{F}_{eVRF}$, $\mathcal{S}$ sends $(\text{Eval}, sid, j, m, R_j)$ for each honest P_j to $\mathcal{A}$.

- (6) Playing the role of $\mathcal{F}_{zk}^{RCL-EC}$, $\mathcal{S}$ sends $(\text{sid}, R_j, \tilde{K}_j, 1)$ for each honest P_j to $\mathcal{A}$.
- (7) Playing the role of $\mathcal{F}_{zk}^{RComKwlg}$, $\mathcal{S}$ sends $(\text{sid}, U_j, 1)$ for each honest P_j to $\mathcal{A}$.
- **Simulating the second step:**
 - (1) Playing the role of $\mathcal{F}_{zk}^{RCL-EC}$, upon receiving $(\text{sid}, R_i, \tilde{K}_i, (\alpha_i, k_i))$ from $\mathcal{A}$ for each $i \in C$, $\mathcal{S}$ checks that $R_i = k_i \cdot E$ and $\tilde{K}_i = \text{Enc}(\alpha_i, k_i)$. If not, $\mathcal{S}$ sends abort to the trusted party computing $\mathcal{F}_{ECDSA}$, simulates the honest parties aborting the execution, and output whatever $\mathcal{A}$ outputs.
 - (2) Playing the role of $\mathcal{F}_{zk}^{RComKwlg}$, upon receiving $(\text{sid}, U_i, (\beta_i, u_i))$ from $\mathcal{A}$ for each $i \in C$, $\mathcal{S}$ checks that $U_i = \text{Com}(\beta_i, u_i)$. If not, $\mathcal{S}$ sends abort to the trusted party computing $\mathcal{F}_{ECDSA}$, simulates the honest parties aborting the execution, and output whatever $\mathcal{A}$ outputs.
 - (3) $\mathcal{S}$ chooses a random value in $\mathbb{Z}_q$ for $k \cdot u$ and sets $w = s \cdot (k \cdot u)$.
 - (4) $\mathcal{S}$ computes $\tilde{Z}_i = \mathcal{H}(m) + r \cdot \tilde{C}_i$ for all $i \in [n]$. For each $i \in [n] \setminus \{j^*\}$, let $\delta'_i = r \cdot \delta_i$. For each $i \in C$, let $x'_i = r \cdot x_i + \mathcal{H}(m)$ (in fact, only a single party adds $\mathcal{H}(m)$).
 - (5) Simulating the scaled decryption of $k \cdot u$:
The simulator $\mathcal{S}$ runs $\mathcal{S}_{sd}^{\text{online}}$ on

$$(\text{sid}, \text{ppCL}, Y^{\text{cl}}, \{U_i\}_{i=1}^n, \{\tilde{K}_i\}_{i=1}^n, \{\alpha_i, \beta_i, k_i, u_i\}_{i \in C}, \{\alpha_i, \beta_i, u_i\}_{i \in [n] \setminus \{j^*\}}, k \cdot u)$$
 to receive v_j for each honest party P_j .
 - (6) Simulating the scaled decryption of $w = u \cdot (H(m) + r \cdot x)$:
The simulator $\mathcal{S}$ runs $\mathcal{S}_{sd}^{\text{online}}$ on

$$(\text{sid}, \text{ppCL}, Y^{\text{cl}}, \{U_i\}_{i=1}^n, \{\tilde{Z}_i\}_{i=1}^n, \{\delta'_i, \beta_i, x'_i, u_i\}_{i \in C}, \{\delta'_i, \beta_i, u_i\}_{i \in [n] \setminus \{j^*\}}, w)$$
 to receive w'_j for each honest party P_j .
 - (7) For each honest party P_j , the simulator $\mathcal{S}$ computes $s'_j = w'_j \cdot v_j^{-1}$ and runs $\text{Verify}(X; m, (r, s' + j))$ to receive β . If $\beta = 0$, then $\mathcal{S}$ sends abort $_j$ to the trusted party computing $\mathcal{F}_{ECDSA}$, and simulates P_j aborting the protocol. Otherwise, it sends s'_j to $\mathcal{F}_{ECDSA}$.
 - (8) The simulator $\mathcal{S}$ outputs whatever $\mathcal{A}$ outputs.

We prove the security by constructing a sequence of hybrids, where the first hybrid is the real execution and the last one is the simulation.

- **Hybrid₀**: The real execution of the protocol.
- **Hybrid₁**: In this hybrid, we consider an imaginary execution with a simulator who plays also the role of $\mathcal{F}_{ECDSA}$, and thus knows the private key and the shares held by the honest parties. Hence, this simulator can run a real execution with $\mathcal{A}$. Clearly, **Hybrid₁** is identical to **Hybrid₀**.
- **Hybrid₂**: This hybrid is identical to the previous hybrid with one change: in the simulation of the scaled decryption, the simulator runs the code of $\mathcal{S}_{sd}^{\text{online}}$. Specifically, it sets $F_{j^*} = a \cdot b \cdot H - \sum_{i \in [n] \setminus \{j^*\}} F_i$.
- **Hybrid₃**: This hybrid is identical to **Hybrid₂** except for the following difference: $\tilde{K}_{j^*}$ and $\tilde{C}_{j^*}$ are set to random elements in their groups.

- **Hybrid₄**: The hybrid is identical to **Hybrid₃** except that U_{j^*} is chosen uniformly from $\hat{G}_q$.
- **Hybrid₅**: The simulation of the signing protocol as described in the text above.

Let us first show that **Hybrid₂** is identical to **Hybrid₁**. The only difference between the two executions is the way F_{j^*} is computed. In **Hybrid₁**, we have

$$\begin{aligned} F_{j^*} &= b_{j^*} \cdot A_2 - \alpha_{j^*} \cdot B + \beta_{j^*} \cdot A_1 \\ &= b_{j^*} \cdot \alpha \cdot Y^{\text{cl}} + b_{j^*} \cdot a \cdot H - \alpha_{j^*} \cdot \beta \cdot G_q - \alpha_{j^*} \cdot b \cdot Y^{\text{cl}} + \beta_{j^*} \cdot \alpha \cdot G_q \\ &= b_{j^*} \cdot a \cdot H + (b_{j^*} \cdot \alpha - \alpha_{j^*} \cdot b) \cdot Y^{\text{cl}} + (\beta_{j^*} \cdot \alpha - \alpha_{j^*} \cdot \beta) \cdot G_q \end{aligned}$$

In contrast, in **Hybrid₂** we have

$$\begin{aligned} F_{j^*} &= ab \cdot H - \sum_{i \in [n] \setminus \{j^*\}} (b_i \cdot A_2 - \alpha_i \cdot B + \beta_i \cdot A_1) \quad (1) \\ &= ab \cdot H - \sum_{i \in [n] \setminus \{j^*\}} b_i \cdot a \cdot H - \sum_{i \in [n] \setminus \{j^*\}} (b_i \cdot \alpha - \alpha_i \cdot b) \cdot Y^{\text{cl}} \\ &\quad - \sum_{i \in [n] \setminus \{j^*\}} (\beta_i \cdot \alpha - \alpha_i \cdot \beta) \cdot G_q \end{aligned}$$

Note that $ab \cdot H - \sum_{i \in [n] \setminus \{j^*\}} b_i \cdot a \cdot H = a \cdot b_{j^*} \cdot H, 0 = \sum_{i \in [n]} (b_i \cdot \alpha - \alpha_i \cdot b) \cdot Y^{\text{cl}}$ which means that $\sum_{i \in [n] \setminus \{j^*\}} (b_i \cdot \alpha - \alpha_i \cdot b) \cdot Y^{\text{cl}} = -b_{j^*} \cdot \alpha - \alpha_{j^*} \cdot b) \cdot Y^{\text{cl}}$ and $0 = \sum_{i \in [n]} (\beta_i \cdot \alpha - \alpha_i \cdot \beta) \cdot G_q$ which means that $\sum_{i \in [n] \setminus \{j^*\}} (\beta_i \cdot \alpha - \alpha_i \cdot \beta) \cdot G_q = -(\beta_{j^*} \cdot \alpha - \alpha_{j^*} \cdot \beta) \cdot G_q$. Hence, by plugging the above into Eq. (2), we obtain exactly the same F_{j^*} as in Hybrid 1.

Next, we claim that **Hybrid₃** is computationally indistinguishable from **Hybrid₂**. The only difference between the two hybrids is P_{j^*} encryptions. In **Hybrid₂** they hold its true shares of x and k , whereas in **Hybrid₃** they are encryptions of random values. However, by the CPA-security of the encryption scheme (which relies on the HSM assumption), the two executions are indistinguishable. Note that a standard reduction to the CPA security is possible for two reasons. First, in both hybrids the simulator does not use the randomness nor the encrypted value in $\tilde{A}_{j^*}$ in the execution to compute F_{j^*} . Second, the simulator does not need to know the secret key y^{cl} , as the parties do not use their shares of the secret decryption key in the protocol at all. Hence, a distinguisher for the CPA security experiment can use the public key and the encryption it receives from its challenger and simulate the execution as required.

Next, it is straightforward that **Hybrid₃** and **Hybrid₄** are distributed the same. The only difference is in the way U_{j^*} is generated. In **Hybrid₃** U_{j^*} is a statistically hiding commitment to a random value, whereas in **Hybrid₄** it is a random element in the group. From the adversary's point of view, this does not make any difference.

Finally, **Hybrid₄** and **Hybrid₅** are distributed the same. The only difference between the hybrids is the way $u \cdot k$ and $u \cdot (\mathcal{H}(m) + r \cdot x)$ are computed. In **Hybrid₄**, u, k and x are chosen separately, whereas in **Hybrid₅**, the simulator chooses $u \cdot k$ uniformly from $\mathbb{Z}_q$ and set the other values accordingly. Note that since u is chosen uniformly in **Hybrid₄** from $\mathcal{D}_q$, it follows that $u \bmod q$ is uniform in $\mathbb{Z}_q$. Hence, in both hybrids $u \cdot k$ and $u \cdot (\mathcal{H}(m) + r \cdot x)$ are distributed uniformly in $\mathbb{Z}_q$ under the constraint that $u \cdot (\mathcal{H}(m) + r \cdot x) \cdot (u \cdot k)^{-1} = s$ and s is a valid signature. This concludes the proof. $\square$

5 ACHIEVING IDENTIFIABLE ABORT

In this section, we augment our protocol from the previous section to achieve the stronger notion of identifiable abort (IA). That is, when the parties abort the protocol due to a failure, they are able to identify a party who has cheated.

Our approach. To achieve IA, our protocol follows the standard approach of having all messages sent over a broadcast channel, accompanied with a non-interactive ZK proof. As all parties see the same message and proof, each party can verify the proof locally, and so if the proof is rejected, all parties abort while blaming the sender.

Signing protocol. The transformation of our signing protocol to achieve IA is straightforward. Currently, the first round already satisfies the above property, as the messages are sent via $\mathcal{F}_{zk}$. In the second round, when the parties run the scaled decryption protocol, we use the Ver-IA version of the protocol (see Protocol 4.1), which requires the parties to also prove the correctness of their messages in the second round. Note that this is possible since the messages in the protocol are the result of a deterministic computation over values committed to in the first round. Hence, each party can prove the correctness of each message with respect to the encryption and commitment they sent in the first round.

Key-generation protocol. In the key-generation protocol, the only message that is not broadcasted is the private share that each party sends to each of the other parties in the second round. To achieve IA, we need to change the second round such that if the share is incorrect, i.e., if $\text{Vss.Vrfy}()$ outputs 0, then the receiver can blame the sender. Note that this is challenging, since a corrupted receiver can try to falsely blame an honest sender. We suggest two solutions:

- *First solution:* As written in Section 3, we can run a public VSS instead of the current protocol. Specifically, each party generates a pair of CL keys and distributes the public encryption key together with a ZK proof that it knows the secret key. We note that these keys should be independent of the public key generated by $\mathcal{F}_{\text{CLKeyGen}}$, which needs to be uniformly sampled and without any of the parties knowing the secret key. Then, when P_i wish to send the share $s_{i,j}$ to party P_j , it encrypts it using P_j 's key and broadcast the ciphertext together with a proof of plaintext knowledge (i.e., through $\mathcal{F}_{zk}^{\text{RCLKwlg}}$). If the proof is approved, P_j can decrypt and obtain its share. Now, if the $\text{Vss.Share}()$ outputs 0, P_j can accuse P_i by broadcasting $s_{i,j}$ together with a proof that it is encrypted in the ciphertext broadcasted before by P_i . If any of the above proofs is rejected then the sender is immediately identified as a cheater.
If all proofs were accepted, then the parties know that $s_{i,j}$ is the share sent from P_i to P_j . Hence, each party can locally run $\text{Vss.Vrfy}(s_{i,j}, \{A_{k,i}\}_{k=0}^t)$ to receive β . If $\beta = 0$, then the parties conclude that the share is inconsistent with the polynomial coefficients published in the exponent by P_i , and so identify P_i as the cheater. Otherwise, $\beta = 1$, and in this case, the parties conclude that P_j falsely accused P_i , and so P_j is identified as the cheater.
- *Second solution:* In this solution, we leverage the simple observation that if P_j accuses P_i then one of the parties must be corrupt. This implies that anyway the share $s_{i,j}$ is known to the adversary.

Hence, in our protocol, if P_j claims that the share sent privately by P_i is inconsistent with the in-the-exponent polynomial coefficients, then we can simply ask P_i to broadcast $s_{i,j}$ to all parties. Then, the parties can verify the consistency of the share and accuse P_i if it is inconsistent. Otherwise, they proceed with the computation. Note that in this case, concluding that P_j is the cheater is not necessarily true, since it is possible that P_i sent a different share to P_j , which is not the same as the share it broadcasted later to the parties.

Security. We need to show that the protocol securely realizes the ideal functionality $\mathcal{F}_{\text{ECDSA}}^{\text{IA}}$ defined in Section 2.2. The proof of security is identical to our proof in Section 4.3, with one minor difference: each time the adversary sends $\mathcal{F}_{zk}$ a message, then the simulator, playing the role of the $\mathcal{F}_{zk}$ ideal functionality, checks the correctness of the statement for each corrupted party, and if it is not correct for some corrupted party P_i , then the simulator sends (abort, i) to the trusted party computing $\mathcal{F}_{\text{ECDSA}}^{\text{IA}}$.

For the steps which would be added to the key generation protocol by the second solution described above, note that the simulator only needs to follow the honest parties' instructions. This holds since it knows all of the data required for the simulation.

REMARK 5.1 (PUBLIC VERIFIABILITY). *In fact, our protocol achieves a stronger notion than standard IA: an external party with the context and view of the transcript is able to identify all faulty participants on abort, or obtain the signature upon success. In the bulletin-board model where each participant publicly posts a message of signed origin (e.g., working on a blockchain), this implies public verifiability.*

6 IMPLEMENTATION AND BENCHMARK RESULTS

We implement a suite of libraries, culminating into this protocol, in Rust¹. Rust is a memory-safe programming language resilient to common programming mistakes, making it a frequent choice for security-focused/cryptographic software.

6.1 Class Groups

We contribute yet another implementation of class groups, this time primarily as an API. This API allows the developer to choose which class group library to instantiate their program with. We provide a pure-Rust option, simplifying the build pipeline and benefiting users targeting the web/embedded devices, and bind BICYCL ([5]) for the most efficient option.

We additionally put forth an experimental, constant-time backend for the composition and reduction of binary quadratic forms. This enables a side-channel resistant prover whose timing is independent of their secrets, reducing the likelihood of leaking bits of the witness (secrets such as private keys). This implementation is primarily benefited from a pre-existing implementation of the optimized Extended Binary GCD ([32]) and from our own implementation of the Binary Reduction Algorithm for binary quadratic forms ([16]).

Independent of backend, algorithms for creating tables to accelerate scaling elements, multi-exponentiation, and compression of elements ([17]) are provided.

¹<https://github.com/kayabaNerve/trout>

	Constant-Time	2-of-3	34-of-50	67-of-100
Time for All Steps	✗	959ms	1,889ms	2,664ms
Time for All Steps	✓	149,259ms	149,786ms	150,538ms
Time for All Steps w/o Identifiable Aborts	✗	689ms	1,531ms	2,333ms
Time for All Steps w/o Identifiable Aborts	✓	62,474ms	63,297ms	64,091ms
eVRF Invocation	✓	344ms	344ms	344ms
eVRFs' Batch Verification	–	23ms	89ms	159ms
Round One's Class Group Proofs Prove	✗	178ms	178ms	178ms
Round One's Class Group Proofs Prove	✓	44,899ms	44,899ms	44,899ms
Round One's Class Group Proofs Batch Verification	–	40ms	556ms	1085ms
Round Two's Proofs Prove	✗	339ms	339ms	339ms
Round Two's Proofs Prove	✓	86,447ms	86,447ms	86,447ms

Table 2: The benchmarks of our implementation, and various specific steps. Not all steps are individually benchmarked, solely the ones which represent a notable amount of the runtime. The "Constant-Time" column denotes if this line-item executes in time independent to the witness, with "–" representing a lack of witness to be independent of. The further columns represent various configurations of required parties, total parties, with the median time taken per-participant (across all applicable runs) in milliseconds. The time taken for proving proofs is duplicated across configurations as it's constant regardless of the threshold. Verification of round two's proofs is omitted as it is not incurred for successful executions of the protocol, and is of comparable complexity to round one's proofs over the class group.

6.2 eVRF

We implement the DDH-premised construction described in [4], slightly modified. The paper notes the ability to use two-bit windows, yet we find greater efficiency with three-bit windows. We also don't perform the bit decomposition of the scalar per invocation, yet once per setup. These two optimizations reduce the size of the Bulletproofs Inner-Product Argument to under 1024 rows when the source group has a 256-bit order, and enables uniformly sampling the eVRF private key (instead of sampling from the suggested approximation $[2^{\log_2(\min(s,q))} - 1]$, so the private key is mutual to the scalar fields of both groups).

The prover executes in constant-time and the verifier is highly benefited from the implemented batch verification.

A future improvement would likely be to use the divisor-premised construction included in an update to [4], which achieves a notably smaller witness while still relying on the DDH assumption, or the Paillier-premised construction which experimentally has been demonstrated to prove in under 100 milliseconds by one of the authors of [4].

6.3 Protocol

The protocol itself is modular to the class-group backend used for the prover (allowing using a constant-time backend), the class-group backend used for the verifier (allowing using the most efficient backend), and the elliptic curve (allowing usage with any elliptic curve, such as the NIST-standardized P-256 or SECG-standardized secp256k1).

For the proofs over the class group, we again allow a modular choice. We provide an implementation of [14] however. The implementation is optimistic in that the round two proofs are only verified if the output signature is invalid, and as prior noted, the round two proofs may be forgone entirely in favor of efficiency.

6.4 Experimental Results

We provide the median per-participant runtime of our protocol across 10 runs (each performing an independent setup with a randomly chosen class group) in Table 2. This is when parameterized to target 128-bit security, including our proofs targeting a 2^{-128} statistical distance from uniform. This manifests in a 1828-bit discriminant for the class group with discriminant Δ_k , causing a 2340-bit discriminant for the class group with discriminant Δ_p operated over (in the notation of [11]). We also provide a more detailed view into some specific steps to provide insight into where the protocol spends its time.

All benchmarks were collected on a Mac mini with a 10-core M2 Pro and 16 GB of RAM. Each view of the signing protocol only ran on a single thread and did not take advantage of any parallelism.

The median amount of bytes transmitted by each participant is 6345 bytes, or 3902 bytes when the identifiable abort functionality is omitted.

Future improvements would be to perform the optimization noted in [11] Section 4.2, transmitting elements of the class group as elements of the class group over Δ_k (lifting them into the larger class group over Δ_p after receiving them) when possible.

ACKNOWLEDGEMENTS

Hila Dahari-Garbian is supported in part by grants from the Israel Science Foundation (no. and 2686/20), by the Simons Foundation Collaboration on the Theory of Algorithmic Fairness and by the Israeli Council for Higher Education (CHE) via the Weizmann Data Science Research Center. Work done in part while a visitor at the FACT Research Center at Reichman University, supported in part by AFOSR Award FA9550-21-1-0046.

REFERENCES

- [1] Damiano Abram, Ariel Nof, Claudio Orlandi, Peter Scholl, and Omer Shlomovits. 2022. Low-Bandwidth Threshold ECDSA via Pseudorandom Correlation Generators. In *IEEE Symposium on Security and Privacy, SP*. 2554–2572.

- [2] Michael Adjedj, Constantin Blokh, Geoffroy Couteau, Antoine Joux, and Nikolaos Makriyannis. 2024. Two-Round 2PC ECDSA at the Cost of 1 OLE. *IACR Cryptol. ePrint Arch.*, paper 2024/1950 (2024).
- [3] Dan Boneh, Rosario Gennaro, and Steven Goldfeder. 2017. Using Level-1 Homomorphic Encryption to Improve Threshold DSA Signatures for Bitcoin Wallet Security. In *LATINCRYPT*. 352–377.
- [4] Dan Boneh, Iftach Haitner, Yehuda Lindell, and Gil Segev. 2025. Exponent-VRFs and Their Applications. In *Advances in Cryptology - EUROCRYPT*. 195–224.
- [5] Cyril Bouvier, Guilhem Castagnos, Laurent Imbert, and Fabien Laguillaumie. 2022. I want to ride my BICYCL: BICYCL Implements CryptographY in CClass groups. *IACR Cryptol. ePrint Arch.*, paper 2022/1466 (2022).
- [6] Lennart Braun, Ivan Damgård, and Claudio Orlandi. 2023. Secure Multiparty Computation from Threshold Encryption Based on Class Groups. In *Advances in Cryptology - CRYPTO*. 613–645.
- [7] Ran Canetti. 2000. Security and Composition of Multiparty Cryptographic Protocols. *J. Cryptol.* 13, 1 (2000), 143–202.
- [8] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. 2020. UC Non-Interactive, Proactive, Threshold ECDSA with Identifiable Aborts. In *ACM CCS*. 1769–1787.
- [9] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. 2019. Two-Party ECDSA from Hash Proof Systems and Efficient Instantiations. In *Advances in Cryptology - CRYPTO*. 191–221.
- [10] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. 2023. Bandwidth-efficient threshold ECDSA revisited: Online/offline extensions, identifiable aborts proactive and adaptive security. *Theor. Comput. Sci.* (2023), 78–104.
- [11] Guilhem Castagnos and Fabien Laguillaumie. 2015. Linearly Homomorphic Encryption from DDH. In *Topics in Cryptology - CT-RSA*. 487–505.
- [12] Guilhem Castagnos, Fabien Laguillaumie, and Ida Tucker. 2018. Practical Fully Secure Unrestricted Inner Product Functional Encryption Modulo p . In *ASIACRYPT*. 733–764.
- [13] Coinbase. 2025. www.coinbase.com
- [14] Handong Cui, Kwan Yin Chan, Tsz Hon Yuen, Xin Kang, and Cheng-Kang Chu. 2024. Bandwidth-Efficient Zero-Knowledge Proofs For Threshold ECDSA. *Comput. J.* (2024), 1265–1278.
- [15] Ivan Damgård and Eiichiro Fujisaki. 2002. A Statistically-Hiding Integer Commitment Scheme Based on Groups with Hidden Order. In *ASIACRYPT*. 125–142.
- [16] Nicolas David, Thomas Espitau, and Akinori Hosoyamada. 2022. Quantum binary quadratic form reduction. *IACR Cryptol. ePrint Arch.*, paper 2022/466 (2022).
- [17] Samuel Dobson, Steven D. Galbraith, and Benjamin Smith. 2020. Trustless unknown-order groups. *IACR Cryptol. ePrint Arch.*, paper 2020/196 (2020).
- [18] Jack Doerner, Yashvanth Kondi, Eysa Lee, and Abhi Shelat. 2018. Secure Two-party Threshold ECDSA from ECDSA Assumptions. In *IEEE Symposium on Security and Privacy, SP*. 980–997.
- [19] Jack Doerner, Yashvanth Kondi, Eysa Lee, and Abhi Shelat. 2019. Threshold ECDSA from ECDSA Assumptions: The Multiparty Case. In *IEEE Symposium on Security and Privacy, SP 2019*. 1051–1066.
- [20] Jack Doerner, Yashvanth Kondi, Eysa Lee, and Abhi Shelat. 2024. Threshold ECDSA in Three Rounds. In *IEEE Symposium on Security and Privacy, SP*. 3053–3071.
- [21] Paul Feldman. 1987. A Practical Scheme for Non-interactive Verifiable Secret Sharing. In *Annual Symposium on Foundations of Computer Science*. 427–437.
- [22] Amos Fiat and Adi Shamir. 1986. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology - CRYPTO*, Andrew M. Odlyzko (Ed.). 186–194.
- [23] Fireblocks. 2025. www.fireblocks.com
- [24] Marc Fischlin. 2005. Communication-Efficient Non-interactive Proofs of Knowledge with Online Extractors. In *Advances in Cryptology - CRYPTO*, Victor Shoup (Ed.). 152–168.
- [25] Ofir Friedman, Avichai Marmor, Dolev Mutzari, Omer Sadika, Yehonatan C. Scaly, Yuval Spiizer, and Avishay Yanai. 2024. 2PC-MPC: Emulating Two Party ECDSA in Large-Scale MPC. *IACR Cryptol. ePrint Arch.*, paper 2024/253 (2024).
- [26] Rosario Gennaro and Steven Goldfeder. 2018. Fast Multiparty Threshold ECDSA with Fast Trustless Setup. In *ACM CCS*. 1179–1194.
- [27] Rosario Gennaro, Steven Goldfeder, and Arvind Narayanan. 2016. Threshold-Optimal DSA/ECDSA Signatures and an Application to Bitcoin Wallet Security. In *Applied Cryptography and Network Security, ACNS*. 156–174.
- [28] Oded Goldreich. 2004. *The Foundations of Cryptography - Volume 2: Basic Applications*. Cambridge University Press.
- [29] Iftach Haitner, Yehuda Lindell, Ariel Nof, and Samuel Ranellucci. 2018. Fast Secure Multiparty ECDSA with Practical Distributed Key Generation and Applications to Cryptocurrency Custody. *IACR Cryptol. ePrint Arch.* (2018), 987.
- [30] Yehuda Lindell. 2017. Fast Secure Two-Party ECDSA Signing. In *Advances in Cryptology - CRYPTO*. 613–644.
- [31] Yehuda Lindell and Ariel Nof. 2018. Fast Secure Multiparty ECDSA with Practical Distributed Key Generation and Applications to Cryptocurrency Custody. In *ACM CCS*. 1837–1854.
- [32] Thomas Pornin. 2020. Optimized Binary GCD for Modular Inversion. *IACR Cryptol. ePrint Arch.*, paper 2020/972 (2020).
- [33] Adi Shamir. 1979. How to Share a Secret. *Commun. ACM* 22, 11 (1979), 612–613.
- [34] Harry W. H. Wong, Jack P. K. Ma, and Sherman S. M. Chow. 2024. Secure Multiparty Computation of Threshold Signatures Made More Efficient. In *Annual Network and Distributed System Security Symposium, NDSS*.
- [35] Harry W. H. Wong, Jack P. K. Ma, Hoover H. F. Yin, and Sherman S. M. Chow. 2023. Real Threshold ECDSA. In *Annual Network and Distributed System Security Symposium, NDSS*.
- [36] Haiyang Xue, Man Ho Au, Mengling Liu, Kwan Yin Chan, Handong Cui, Xiang Xie, Tsz Hon Yuen, and Chengru Zhang. 2023. Efficient Multiplicative-to-Additive Function from Joye-Libert Cryptosystem and Its Application to Threshold ECDSA. In *ACM CCS*. 2974–2988.
- [37] Haiyang Xue, Man Ho Au, Xiang Xie, Tsz Hon Yuen, and Handong Cui. 2021. Efficient Online-friendly Two-Party ECDSA Signature. In *ACM CCS*. 558–573.
- [38] Tsz Hon Yuen, Handong Cui, and Xiang Xie. 2021. Compact Zero-Knowledge Proofs for Threshold ECDSA with Trustless Setup. In *Public-Key Cryptography - PKC*. 481–511.
- [39] Zengo. 2025. www.zengo.com

A ZERO-KNOWLEDGE PROOF OF KNOWLEDGE PROTOCOL

In this section, we show how to instantiate the $\mathcal{F}_{zk}^R$ ideal functionality for an arbitrary relation R . In particular, we generalize the proofs within [14] (which is an improvement of [38]) to offer a comprehensive proof for arbitrary relations. These proofs have been shown to be secure under the Adaptive Root Assumption and Hard Subgroup Membership Assumption. To obtain a non-interactive protocol, it is possible to apply the Fiat-Shamir transform [22]. However, there is no evidence that the resulting protocol is UC-secure. As an alternative, one can use the transformation of Fischlin [24] which offers non-rewinding extraction and hence enables UC-security.

We define the function $\text{ord}(\cdot)$ that takes an element as its input and return its order (in our setting, q or unknown).

We define the relation

$$\mathcal{R} = \left\{ \begin{array}{l} \vec{M} \in (\hat{G} \vee \hat{E})^{m \times n}, \quad \vec{O} \in (\hat{G} \vee \hat{E})^m, \quad \vec{w} \\ \text{s.t.} \\ \forall i \in [m] : O_i = \sum_{j=1}^n w_j \cdot M_{i,j} \\ \forall i \in [m] : (\forall j \in [n] : M_{i,j} \in \hat{G} \cup \{0\}) \vee (\forall j \in [n] : M_{i,j} \in \hat{E} \cup \{0\}) \\ \forall i \in [m] \forall j \in [n] : \text{ord}(M_{i,j}) = \text{ord}(w_j) \vee M_{i,j} = 0 \end{array} \right\}$$

The second requirement only means that all elements in each row are from the same group, whereas the third requirement states that each two elements that are multiplied together should have the same order.

Let $\vec{M}_i$ be the i th row of $\vec{M}$. In the protocol we make use of the following notation:

- $I_G \subseteq [m] = \{i \in [m] : \vec{M}_i \in \hat{G} \cup \{0\}\}$
(indices of rows in $\vec{M}$ that contains elements from $\hat{G}$ or are 0).
- $I_E \subseteq [m] = \{i \in [m] : \vec{M}_i \in \hat{E} \cup \{0\}\}$
(indices of rows in $\vec{M}$ that contains elements from $\hat{E}$ or 0).
- $J_u \subseteq [n] = \{j \in [n] : \text{ord}(w_j) \text{ unknown}\}$
(indices of elements in $\vec{w}$ with unknown order).
- $J_q \subseteq [n] = \{j \in [n] : \text{ord}(w_j) = q\}$
(indices of elements in $\vec{w}$ with order q).

The protocol proceeds as follows:

- **Public input:** λ , ppCL, ppEC, $\vec{M}$, $\vec{O}$, B .
- **Prover's input:** $\vec{w}$
- **The protocol:**

- (1) **Prover's first message:** The prover samples random $\vec{r} = r_1, \dots, r_n$ as follows: if $\text{ord}(w_j) = q$, then r_j is chosen from $\mathbb{Z}_q$, whereas if $\text{ord}(w_j)$ is unknown, then r_j is chosen from $[-B, B]$.

Then, it computes $\vec{R} = \vec{r} \cdot \vec{M}$ and sends $\vec{R}$ to the verifier.

- (2) **Verifier's challenge:** The verifier samples random challenges $c \in \mathbb{Z}_q$, and a prime $\ell \in \text{Primes}(\lambda)$ and sends it to the prover.

- (3) **Prover's second message:** The prover works by the following instructions:

- (a) $\forall j \in [n]$ compute $s_j = r_j + c \cdot w_j$.
- (b) $\forall j \in J_u$: find $d_j \in \mathbb{Z}, e_j \in [0, \dots, q \cdot \ell - 1]$ such that $s_j = d_j \cdot q \cdot \ell + e_j$.
- (c) $\forall i \in I_G$ compute $D_i = \sum_{j=1}^n d_j \cdot M_{i,j}$

The prover sends $\{s_j\}_{j \in J_q}, \{D_i\}_{i \in [m]}, \{e_j\}_{j \in J_u}$

- (4) **Verifier's decision:** The verifier checks that

- (a) $e_j \in [0, \dots, q \cdot \ell - 1]$
- (b) $\forall i \in I_E : \sum_{j=1}^n s_j \cdot M_{i,j} = R_i + c \cdot O_i$

$$(c) \quad \forall i \in I_G : \ell \cdot q \cdot D_i + \sum_{j=1}^n e_j \cdot M_{i,j} = R_i + c \cdot O_i$$

If the equations does not hold, the verifier outputs reject. Otherwise, it outputs accept.

Completeness. Assume that the prover is honest. Then, for each $i \in [I_E]$

$$\sum_{j=1}^n s_j \cdot M_{i,j} = \sum_{j=1}^n (r_j + c \cdot w_j) M_{i,j} = \sum_{j=1}^n r_j \cdot M_{i,j} + c \cdot \sum_{j=1}^n w_j \cdot M_{i,j} = R_i + c \cdot O_i$$

as required. Next, we have that for each $i \in I_G$

$$q\ell \cdot D_i + \sum_{j=1}^n e_j \cdot M_{i,j} = \sum_{j=1}^n (q\ell \cdot d_j + e_j) \cdot M_{i,j} = \sum_{j=1}^n s_j \cdot M_{i,j}$$

which is equal to

$$R_i + c \cdot O_i = \sum_{j=1}^n (r_j + c \cdot w_j) M_{i,j} = \sum_{j=1}^n s_j \cdot M_{i,j}$$

since by construction $s_j = d_j \cdot q\ell + e_j = r_j + c \cdot w_j$. Hence, both checks will pass successfully and the verifier will output accept.